Python: exercises on dictionaries, tuples, and sets

This notebook was generated in solutions mode.

Exercise 1: Merge Two Dictionaries

Exercise Description

Create a function `merge_dicts(dict1, dict2)` that combines two dictionaries. If a key exists in both, sum their values.

Your solution

```
def merge_dicts(dict1, dict2):
    # Create a copy to avoid modifying the original dictionary
    result = dict1.copy()
```

```
    # Iterate through the second dictionary
    for key, value in dict2.items():
        if key in result:
            # If key exists, sum the values
            result[key] = result[key] + value
        else:
            # If key doesn't exist, add it
            result[key] = value

    return result

# Example usage
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
print(merge_dicts(dict1, dict2))  # Output: {'a': 1, 'b': 5, 'c': 4}
```

Test your solution

```
# Test case 1
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
result = merge_dicts(dict1, dict2)
print(result)  # Expected: {'a': 1, 'b': 5, 'c': 4}
```

Test your solution

```
# Test case 2
dict1 = {'x': 10}
dict2 = {'y': 20, 'x': 5}
```

```
result = merge_dicts(dict1, dict2)
print(result)  # Expected: {'x': 15, 'y': 20}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 2: Reverse Lookup

Exercise Description

Write a function `reverse_lookup(d, value)` that returns all keys from dictionary `d` that map to the given `value`.

Your solution

```
def reverse_lookup(d, value):
    # Initialize list to store matching keys
    result = []

    # Iterate through dictionary items
```

```
    for key, val in d.items():
        if val == value:
            # Add key to result if value matches
            result.append(key)

    return result

# Example usage
d = {'a': 1, 'b': 2, 'c': 1}
print(reverse_lookup(d, 1))  # Output: ['a', 'c']
```

Test your solution

```
# Test case 1
d = {'a': 1, 'b': 2, 'c': 1}
result = reverse_lookup(d, 1)
print(result)  # Expected: ['a', 'c']
```

Test your solution

```
# Test case 2
d = {'x': 'hello', 'y': 'world', 'z': 'hello'}
result = reverse_lookup(d, 'hello')
print(result)  # Expected: ['x', 'z']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 3: Count Tuple Elements

Exercise Description

Write a function `count_elements(tup)` that returns a dictionary with counts of each element in the tuple `tup`.

Your solution

```
def count_elements(tup):
    # Initialize dictionary to store counts
    counts = {}

    # Iterate through tuple elements
    for element in tup:
        if element in counts:
            # Increment count if element exists
            counts[element] += 1
        else:
```

```
            # Initialize count to 1 if element is new
            counts[element] = 1

    return counts

# Example usage
tup = (1, 2, 2, 3, 1)
print(count_elements(tup))  # Output: {1: 2, 2: 2, 3: 1}
```

Test your solution

```
# Test case 1
tup = (1, 2, 2, 3, 1)
result = count_elements(tup)
print(result)  # Expected: {1: 2, 2: 2, 3: 1}
```

Test your solution

```
# Test case 2
tup = ('a', 'b', 'a', 'c', 'b', 'a')
result = count_elements(tup)
print(result)  # Expected: {'a': 3, 'b': 2, 'c': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 4: Set Intersection

Exercise Description

Implement a function `intersect(s1, s2)` that returns the intersection of two sets `s1` and `s2`.

Your solution

```
def intersect(s1, s2):
    # Method 1: Using set intersection operator
    return s1 & s2

    # Method 2: Manual implementation
    # result = set()
    # for element in s1:
    #     if element in s2:
    #         result.add(element)
    # return result

# Example usage
s1 = {1, 2, 3}
s2 = {2, 3, 4}
print(intersect(s1, s2))  # Output: {2, 3}
```

Test your solution

```
# Test case 1
s1 = {1, 2, 3}
s2 = {2, 3, 4}
result = intersect(s1, s2)
print(result)  # Expected: {2, 3}
```

Test your solution

```
# Test case 2
s1 = {'a', 'b', 'c'}
s2 = {'b', 'c', 'd'}
result = intersect(s1, s2)
print(result)  # Expected: {'b', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 5: Remove Duplicates

Exercise Description

Write a function `remove_duplicates(lst)` that returns a list without any duplicates using a set.

Your solution

```
def remove_duplicates(lst):
    # Convert to set to remove duplicates, then back to list
    return list(set(lst))

    # Alternative method preserving order:
    # seen = set()
    # result = []
    # for item in lst:
    #     if item not in seen:
    #         seen.add(item)
    #         result.append(item)
    # return result

# Example usage
lst = [1, 2, 2, 3, 4, 4, 5]
print(remove_duplicates(lst))  # Output: [1, 2, 3, 4, 5] (order may vary)
```

Test your solution

```
# Test case 1
lst = [1, 2, 2, 3, 4, 4, 5]
result = remove_duplicates(lst)
print(result)  # Expected: [1, 2, 3, 4, 5]
```

Test your solution

```
# Test case 2
lst = ['a', 'b', 'a', 'c', 'b']
result = remove_duplicates(lst)
print(result)  # Expected: ['a', 'b', 'c'] (order may vary)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 6: Tuple to Dictionary

Exercise Description

Create a function `tuple_to_dict(tup)` that turns a tuple of `(key, value)` pairs into a dictionary.

Your solution

```
def tuple_to_dict(tup):
    # Method 1: Using dict() constructor
    return dict(tup)

    # Method 2: Manual implementation
    # result = {}
    # for key, value in tup:
    #     result[key] = value
    # return result

# Example usage
tup = (('a', 1), ('b', 2))
print(tuple_to_dict(tup))  # Output: {'a': 1, 'b': 2}
```

Test your solution

```
# Test case 1
tup = (('a', 1), ('b', 2))
result = tuple_to_dict(tup)
print(result)  # Expected: {'a': 1, 'b': 2}
```

Test your solution

```
# Test case 2
tup = (('x', 10), ('y', 20), ('z', 30))
result = tuple_to_dict(tup)
print(result)  # Expected: {'x': 10, 'y': 20, 'z': 30}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 7: Value Switch in Dictionary

Exercise Description

Write a function `switch_values(d)` that switches the keys and values in a dictionary. Assume all values are unique.

Your solution

```python
def switch_values(d):
    # Create new dictionary with keys and values switched
    return {value: key for key, value in d.items()}

    # Alternative manual implementation:
    # result = {}
    # for key, value in d.items():
    #     result[value] = key
    # return result

# Example usage
d = {'a': 1, 'b': 2}
print(switch_values(d))  # Output: {1: 'a', 2: 'b'}
```

Test your solution

```python
# Test case 1
d = {'a': 1, 'b': 2}
result = switch_values(d)
print(result)  # Expected: {1: 'a', 2: 'b'}
```

Test your solution

```python
# Test case 2
d = {'name': 'John', 'age': 25}
result = switch_values(d)
print(result)  # Expected: {'John': 'name', 25: 'age'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 8: Dictionary of Squares

Exercise Description

Create a function `squares(n)` that returns a dictionary of numbers from 1 to `n` where the keys are numbers and the values are their squares.

Your solution

```
def squares(n):
    # Method 1: Using dictionary comprehension
    return {i: i**2 for i in range(1, n+1)}

    # Method 2: Manual implementation
    # result = {}
    # for i in range(1, n+1):
```

```
    #     result[i] = i * i
    # return result

# Example usage
print(squares(5))  # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Test your solution

```
# Test case 1
result = squares(5)
print(result)  # Expected: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Test your solution

```
# Test case 2
result = squares(3)
print(result)  # Expected: {1: 1, 2: 4, 3: 9}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 9: Set Difference

Exercise Description

Write a function `difference(s1, s2)` that returns a set of elements that are only in the first set `s1` but not in the second set `s2`.

Your solution

```
def difference(s1, s2):
    # Method 1: Using set difference operator
    return s1 - s2

    # Method 2: Manual implementation
    # result = set()
    # for element in s1:
    #     if element not in s2:
    #         result.add(element)
    # return result

# Example usage
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print(difference(s1, s2))  # Output: {1, 2}
```

Test your solution

```
# Test case 1
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
result = difference(s1, s2)
print(result)  # Expected: {1, 2}
```

Test your solution

```
# Test case 2
s1 = {'a', 'b', 'c'}
s2 = {'b', 'd'}
result = difference(s1, s2)
print(result)  # Expected: {'a', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 10: Count Vowels

Exercise Description

Create a function `count_vowels(s)` that counts and returns a dictionary with vowels as keys and their counts in the string `s` as values.

Your solution

```
def count_vowels(s):
    # Define vowels
    vowels = set('aeiou')
    counts = {}

    # Iterate through string characters
    for char in s.lower():
        if char in vowels:
            if char in counts:
                counts[char] += 1
            else:
                counts[char] = 1

    return counts

# Example usage
print(count_vowels('hello world'))  # Output: {'e': 1, 'o': 2}
```

Test your solution

```
# Test case 1
result = count_vowels('hello world')
print(result)  # Expected: {'e': 1, 'o': 2}
```

Test your solution

```
# Test case 2
result = count_vowels('programming')
print(result)  # Expected: {'o': 1, 'a': 1, 'i': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 11: Find Max Value

Exercise Description

Write a function `find_max(d)` that returns the key with the maximum value in the dictionary `d`.

Your solution

```
def find_max(d):
    # Method 1: Using max() with key parameter
    return max(d, key=d.get)

    # Method 2: Manual implementation
    # max_key = None
    # max_value = float('-inf')
    # for key, value in d.items():
    #     if value > max_value:
    #         max_value = value
    #         max_key = key
    # return max_key

# Example usage
d = {'a': 5, 'b': 12, 'c': 3}
print(find_max(d))  # Output: 'b'
```

Test your solution

```
# Test case 1
d = {'a': 5, 'b': 12, 'c': 3}
result = find_max(d)
print(result)  # Expected: 'b'
```

Test your solution

```
# Test case 2
d = {'x': 100, 'y': 50, 'z': 200}
result = find_max(d)
print(result)  # Expected: 'z'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 12: Unique Characters

Exercise Description

Create a function `unique_chars(s)` that returns a set of unique characters in the string `s`.

Your solution

```
def unique_chars(s):
    # Convert string to set to get unique characters
    return set(s)
```

```
# Example usage
print(unique_chars('hello'))  # Output: {'e', 'h', 'l', 'o'}
```

Test your solution

```
# Test case 1
result = unique_chars('hello')
print(result)  # Expected: {'e', 'h', 'l', 'o'}
```

Test your solution

```
# Test case 2
result = unique_chars('programming')
print(result)  # Expected: {'p', 'r', 'o', 'g', 'a', 'm', 'i', 'n'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 13: Key Multiplication

Exercise Description

Write a function `multiply_keys(d, factor)` that returns a new dictionary where each key is multiplied by a `factor`.

Your solution

```
def multiply_keys(d, factor):
    # Create new dictionary with keys multiplied by factor
    return {key * factor: value for key, value in d.items()}

    # Alternative manual implementation:
    # result = {}
    # for key, value in d.items():
    #     result[key * factor] = value
    # return result

# Example usage
d = {1: 100, 2: 200}
print(multiply_keys(d, 2))  # Output: {2: 100, 4: 200}
```

Test your solution

```
# Test case 1
d = {1: 100, 2: 200}
result = multiply_keys(d, 2)
print(result)  # Expected: {2: 100, 4: 200}
```

Test your solution

```
# Test case 2
d = {3: 'a', 5: 'b'}
result = multiply_keys(d, 10)
print(result)  # Expected: {30: 'a', 50: 'b'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 14: Set Complement

Exercise Description

Implement a function `complement(full_set, sub_set)` that returns the set of elements in `full_set` that are not in `sub_set`.

Your solution

```python
def complement(full_set, sub_set):
    # Method 1: Using set difference operator
    return full_set - sub_set

    # Method 2: Manual implementation
    # result = set()
    # for element in full_set:
    #     if element not in sub_set:
    #         result.add(element)
    # return result

# Example usage
full_set = {1, 2, 3, 4, 5}
sub_set = {2, 4}
print(complement(full_set, sub_set))  # Output: {1, 3, 5}
```

Test your solution

```python
# Test case 1
full_set = {1, 2, 3, 4, 5}
sub_set = {2, 4}
result = complement(full_set, sub_set)
print(result)  # Expected: {1, 3, 5}
```

Test your solution

```python
# Test case 2
full_set = {'a', 'b', 'c', 'd'}
```

```
sub_set = {'b', 'd'}
result = complement(full_set, sub_set)
print(result)  # Expected: {'a', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 15: Count Keys Starting With

Exercise Description

Write a function `count_keys_starting_with(d, char)` that returns the count of keys in dictionary `d` that start with `char`.

Your solution

```
def count_keys_starting_with(d, char):
    # Initialize counter
    count = 0
```

```
    # Iterate through dictionary keys
    for key in d.keys():
        # Convert key to string and check first character
        if str(key).startswith(char):
            count += 1

    return count

# Example usage
d = {'apple': 1, 'banana': 2, 'apricot': 3, 'cherry': 4}
print(count_keys_starting_with(d, 'a'))  # Output: 2
```

Test your solution

```
# Test case 1
d = {'apple': 1, 'banana': 2, 'apricot': 3, 'cherry': 4}
result = count_keys_starting_with(d, 'a')
print(result)  # Expected: 2
```

Test your solution

```
# Test case 2
d = {'hello': 1, 'world': 2, 'python': 3}
result = count_keys_starting_with(d, 'h')
print(result)  # Expected: 1
```

Debug your solution